EE6083 VLSI Testing

Quartz-4b7e - HW1

➢ 1-1 BIST circuit design for Ripple carry adder

1. Full adder implementation

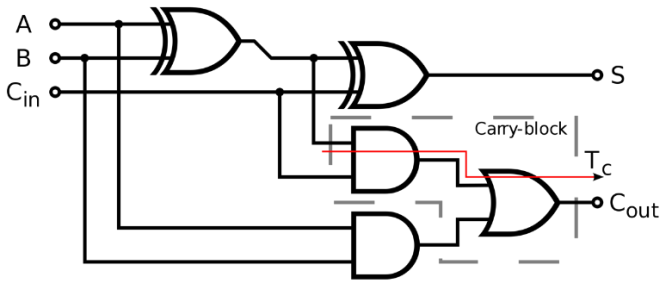


圖 1 Full adder 電路

```
1  `timescale 1ns/1ps
2
3  module full_adder(
4          input in1, in2, Cin,
5          output Sum, Cout
6  );
7
8          wire w1, w2, w3;
9
10         xor u0(w1, in1, in2);
11         xor u1(Sum, Cin, w1);
12         and u2(w2, w1, Cin);
13         and u3(w3, in1, in2);
14         or  u4(Cout, w2, w3);
15
16 endmodule
```

圖 1 Full adder Verilog code

圖 1 為我所參考的 full adder 電路架構，圖 2 為相對應之 Verilog code。

2. RCA with fault

下圖 3 為 ripple carry adder 之 Verilog code。而除了基本 RCA 功能外，我設定當電路為 testing mode 時將第一個 full adder (*fa0*)的 carry 固定為 1，模擬電路發生 single stuck at 1 fault 的情況 (圖中紅色框框)。

```
1  `timescale 1ns/1ps
2
3  module RCA(
4          input [15:0] in1, in2,
5          input Cin, test,
6          output [15:0] Sum,
7          output Cout
8  );
9
10         wire [15:0]carry;
11         wire temp; // for normal output
12         assign Cout = carry[15];
13
14         ////////////////////////////// RCA //////////////////////////////
15
16         /* single stuck at fault occur in carry of full_adder 7 */
17         full_adder fa0 (in1[0],  in2[0],  Cin, Sum[0],  temp);
18         assign carry[0] = test? 1'b1:temp;
19         ////////////////////////////////////////////////////////////////
20
21         full_adder fa1 (in1[1],  in2[1],  carry[0],  Sum[1],  carry[1]);
22         full_adder fa2 (in1[2],  in2[2],  carry[1],  Sum[2],  carry[2]);
23         full_adder fa3 (in1[3],  in2[3],  carry[2],  Sum[3],  carry[3]);
24         full_adder fa4 (in1[4],  in2[4],  carry[3],  Sum[4],  carry[4]);
25         full_adder fa5 (in1[5],  in2[5],  carry[4],  Sum[5],  carry[5]);
26         full_adder fa6 (in1[6],  in2[6],  carry[5],  Sum[6],  carry[6]);
27         full_adder fa7 (in1[7],  in2[7],  carry[6],  Sum[7],  carry[7]);
28         full_adder fa8 (in1[8],  in2[8],  carry[7],  Sum[8],  carry[8]);
29         full_adder fa9 (in1[9],  in2[9],  carry[8],  Sum[9],  carry[9]);
30         full_adder fa10(in1[10], in2[10], carry[9],  Sum[10], carry[10]);
31         full_adder fa11(in1[11], in2[11], carry[10], Sum[11], carry[11]);
32         full_adder fa12(in1[12], in2[12], carry[11], Sum[12], carry[12]);
33         full_adder fa13(in1[13], in2[13], carry[12], Sum[13], carry[13]);
34         full_adder fa14(in1[14], in2[14], carry[13], Sum[14], carry[14]);
35         full_adder fa15(in1[15], in2[15], carry[14], Sum[15], carry[15]);
36         ////////////////////////////////////////////////////////////////
37
38 endmodule
```

Single stuck at 1 fault in testing mode

圖 2 Ripple carry adder Verilog code

3. BIST

```
`timescale 1ns/1ps

module BIST(
        input Reset, clk, En, test, carry_in, FA_carry,
        input [15:0] x_in, y_in, FA_sum,
        output reg pf, FA_carry_in,
        output reg [15:0] x_out, y_out
);

        reg [3:0] counter;

        always@(posedge clk)begin

                if(Reset)begin
                        counter <= 4'd0;
                end

                else begin

                        if( test == 0 )begin
                                counter <= 4'd0;
                        end

                        else begin

                                if( counter < 4'd9 )begin
                                        counter <= counter + 4'd1;
                                end

                                else begin
                                        counter <= counter;
                                end

                        end

                end

        end

        always@(posedge clk)begin

                if(Reset)begin
                        x_out <= 16'd0;
                        y_out <= 16'd0;
                        FA_carry_in <= 1'b0;
                end

                else begin
                        // normal mode //
                        if( test == 1'b0 )begin
                                x_out <= x_in;
                                y_out <= y_in;
                                FA_carry_in <= carry_in;
                        end
                        // testing mode //
                        else if( test == 1'b1 && En == 1'b1 )begin

                                case(counter)

                                4'd0: begin
                                        x_out <= 16'b0000_0000_0000_0000;
                                        y_out <= 16'b0000_0000_0000_0000;
                                        FA_carry_in <= 1'b0;
                                end

                                4'd1: begin
                                        x_out <= 16'b0000_0000_0000_0000;
                                        y_out <= 16'b1111_1111_1111_1111;
                                        FA_carry_in <= 1'b0;
                                end

                                4'd2: begin
                                        x_out <= 16'b0000_0000_0000_0000;
                                        y_out <= 16'b1111_1111_1111_1111;
                                        FA_carry_in <= 1'b1;
                                end

                                4'd3: begin
                                        x_out <= 16'b1111_1111_1111_1111;
                                        y_out <= 16'b0000_0000_0000_0000;
                                        FA_carry_in <= 1'b0;
                                end

                                4'd4: begin
                                        x_out <= 16'b1111_1111_1111_1111;
                                        y_out <= 16'b0000_0000_0000_0000;
                                        FA_carry_in <= 1'b1;
                                end

                                4'd5: begin
                                        x_out <= 16'b1111_1111_1111_1111;
                                        y_out <= 16'b1111_1111_1111_1111;
                                        FA_carry_in <= 1'b1;
                                end

                                4'd6: begin
                                        x_out <= 16'b0101_0101_0101_0101;
                                        y_out <= 16'b0101_0101_0101_0101;
                                        FA_carry_in <= 1'b1;
                                end

                                4'd7: begin
                                        x_out <= 16'b1010_1010_1010_1010;
                                        y_out <= 16'b1010_1010_1010_1010;
                                        FA_carry_in <= 1'b0;
                                end

                                endcase

                        end

                        else begin
                        end

                end
        end

        always@(posedge clk)begin

                if(Reset)begin
                        pf <= 1'bz;
                end

                else begin

                        if( test == 1'b0 )begin
                                pf <= 1'bz;
                        end

                        else begin
                                case(counter)

                                4'd1: begin

                                        if( FA_carry == 1'b0 && FA_sum == 16'b0000_0000_0000_0000 )begin
                                                pf <= 1'b0;
                                        end
                                        else begin
                                                pf <= 1'b1;
                                        end

                                end

                                4'd2: begin

                                        if( FA_carry == 1'b0 && FA_sum == 16'b1111_1111_1111_1111 )begin
```

圖 4 BIST 電路 Verilog code

```
                                                pf <= 1'b0;
                                        end
                                        else begin
                                                pf <= 1'b1;
                                        end

                                end

                                4'd3: begin

                                        if( FA_carry == 1'b1 && FA_sum == 16'b0000_0000_0000_0000 )begin
                                                pf <= 1'b0;
                                        end
                                        else begin
                                                pf <= 1'b1;
                                        end

                                end

                                4'd4: begin

                                        if( FA_carry == 1'b0 && FA_sum == 16'b1111_1111_1111_1111 )begin
                                                pf <= 1'b0;
                                        end
                                        else begin
                                                pf <= 1'b1;
                                        end

                                end

                                4'd5: begin

                                        if( FA_carry == 1'b1 && FA_sum == 16'b0000_0000_0000_0000 )begin
                                                pf <= 1'b0;
                                        end
                                        else begin
                                                pf <= 1'b1;
                                        end

                                end

                                4'd6: begin

                                        if( FA_carry == 1'b1 && FA_sum == 16'b1111_1111_1111_1111 )begin
                                                pf <= 1'b0;
                                        end
                                        else begin
                                                pf <= 1'b1;
                                        end

                                end

                                4'd7: begin

                                        if( FA_carry == 1'b0 && FA_sum == 16'b1010_1010_1010_1011 )begin
                                                pf <= 1'b0;
                                        end
                                        else begin
                                                pf <= 1'b1;
                                        end

                                end

                                4'd8: begin

                                        if( FA_carry == 1'b1 && FA_sum == 16'b0101_0101_0101_0100 )begin
                                                pf <= 1'b0;
                                        end
                                        else begin
                                                pf <= 1'b1;
                                        end

                                end

                                default: begin

                                        pf <= 1'b0;

                                end

                                endcase

                        end
                end
        end
endmodule
```

圖 5 BIST 電路 Verilog code

圖 4 與圖 5 為 BIST 電路之 Verilog code。在 testing mode 時會依序

輸入 8 個 test pattern，進行 fault 測試，如果電路出現 fault 則下個 clock cycle *pf* 訊號將會為 1 反之則為 0。

4. Main circuit

圖 6 為完整電路的 Verilog code，main 將 RCA 與 BIST 電路接起來，由 *test* 訊號決定要進行 normal mode 還是 testing mode。

```
1  `timescale 1ns/1ps
2
3  module main(
4          input Reset, clk, En, test, carry_in,
5          input [15:0] x, y,
6          output [15:0] Sum,
7          output pf, FA_Cout
8  );
9
10         wire FA_Cin;
11         wire [15:0] FA_x, FA_y;
12
13         BIST bist(Reset, clk, En, test, carry_in, FA_Cout, x, y, Sum, pf, FA_Cin, FA_x, FA_y);
14         RCA rca(FA_x, FA_y, FA_Cin, test, Sum, FA_Cout);
15
16 endmodule
```

圖 6 main circuit Verilog code

5. Testbench & simulation result

圖 7 為 testbench，圖 8 及圖 9 分別為 normal mode 及 testing mode 根據 testbench 模擬出的波形圖。

在模擬波形圖中 0ns ~ 100ns 為 normal mode (圖 8)。從波形中可以看到，當 Reset 完成(20ns 後)並將輸入設定為 16'd300 與 16'd59、carry in 為 0 時，輸出 Sum 為 16'd359；輸入為 16'd15 及 16'd20、carry in 為 1 時，Sum 為 16'd36；輸入為 16'd1300 及 16'd56、carry in 為 1 時 Sum 為 16'd1357；輸入為 16'd50 及 16'd50、carry in 為 0 時 Sum 為 16'd150，可確認 RCA 電路的正確性及 BIST 電路在 normal mode 時能正常將待相加的輸入值完整送入 RCA 中。而此時因為不是 testing mode，*pf* 訊號設定為 high impedance。

```
1  `timescale 1ns/1ps
2
3  module tb();
4
5          reg [15:0] x, y;
6          reg Cin, test, clk, Reset, En;
7          wire [15:0] Sum;
8          wire Cout, pf;
9
10         main u0(Reset, clk, En, test, Cin, x, y, Sum, pf, Cout);
11
12         initial begin
13
14         clk = 1'b1;
15
16         forever begin
17         #10 clk = ~clk;
18         end
19
20         end
21
22         initial begin
23
24         $fsdbDumpfile("HW1_1.fsdb");
25         $fsdbDumpvars(0, tb);
26
27         Reset = 1'b1;
28         Cin = 1'b0;
29         test = 1'b0;
30         En = 1'b0;
31         x = 16'b0000_0000_0000_0000;
32         y = 16'b0000_0000_0000_0000;
33
34         #20     Reset = 1'b0;
35                 x = 16'd300;
36                 y = 16'd 59;
37                 Cin = 1'b0;
38
39         #20     x = 16'd15;
40                 y = 16'd20;
41                 Cin = 1'b1;
42
43         #20     x = 16'd1300;
44                 y = 16'd56;
45                 Cin = 1'b1;
46
47         #20     x = 16'd50;
48                 y = 16'd100;
49                 Cin = 1'b0;
50
51         #20     test = 1'b1;
52                 En =1'b1;
53
54         #240    Reset = 1'b1;
55                 En = 1'b0;
56                 x = 16'bzzzz_zzzz_zzzz_zzzz;
57                 y = 16'bzzzz_zzzz_zzzz_zzzz;
58                 Cin = 1'bz;
59
60         #20     $finish;
61
62         end
63
64 endmodule
```

圖 7 testbench

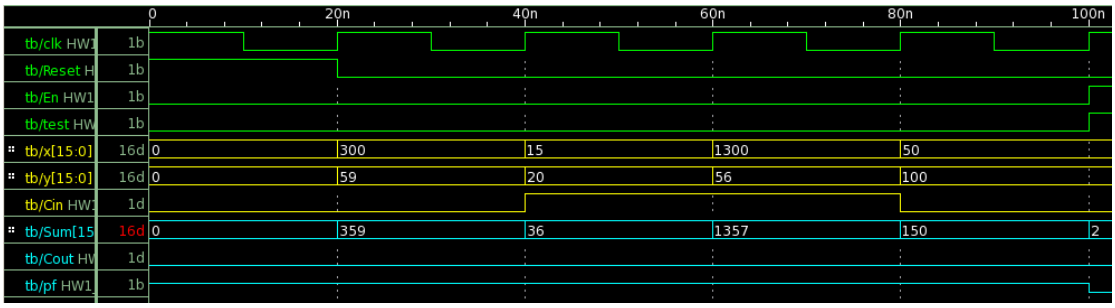


圖 8 Normal mode 波形圖（0ns ~ 100ns）

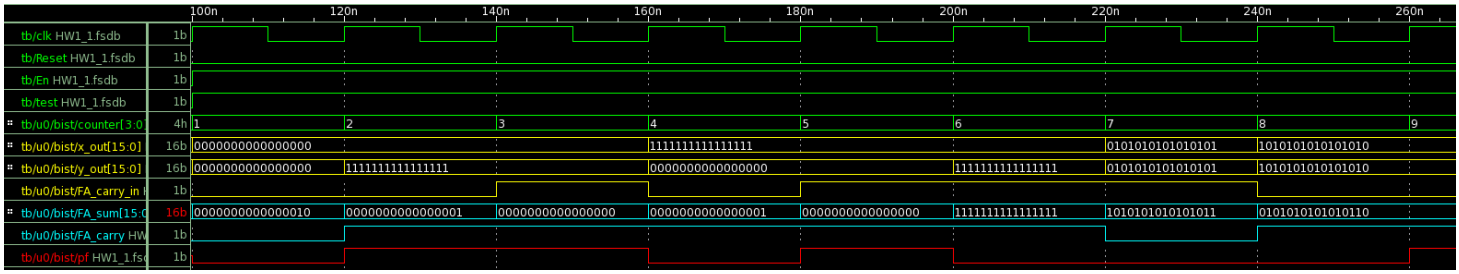


圖 9 Testing mode 波形圖（100ns ~ ）

圖 9 之波形圖為 testing mode， RCA 的輸入將由 BIST 電路提供，依序給 8 組 test pattern 並量測 RCA 輸出正確性。而如同前面所述，為了能夠確認 BIST 之正確性，在 testing mode 時 RCA 中將有一個 full adder 之 carry 將常駐為 1 以模擬 single stuck at fault (選擇 stuck at 1 fault)。以下將分析其中幾個 test pattern 之測試結果:

i. test pattern 為 16'd0 與 16'd0 相加、carry in 為 0，但 Sum 卻是 16'b0000_0000_0000_0010，結果錯誤，因此在下個 clock cycle *pf* 訊號將輸出 1 以表示抓到 fault。

ii. 16'd 0 與 16'b1111_1111_1111_1111 相加、carry in 為 0，此時 Sum 為 16'd1 且 carry out 為 1，結果錯誤因此下個 clock cycle *pf* 訊號將輸出 1。

iii. 16'd 0 與 16'b1111_1111_1111_1111 相加、carry in 為 1，此時 Sum 為 16'd0 且 carry out 為 1，結果正確，因此下個 clock cycle *pf* 訊號將輸出 0，代表電路計算結果正確。

iv. 16'b0101_0101_0101_0101 與 16'b0101_0101_0101_0101 相加、carry in 為 1，此時 Sum 為 16'b1010_1010_1010_1011，carry out 為 0，計算結果正確因此下個 clock cycle *pf* 將會輸出 0。

其他 test pattern 的分析方法相同，如果 RCA 輸出之 Sum 與 carry 正確，則下個 clock cycle *pf* 訊號將輸出 0，反之如果錯誤則輸出 1。

➢ 1-2 List fault list

由上課教過的各種邏輯閘的 fault diagram，可以列出整個電路的 fault diagram (圖 10)，並從中挑選出要留下的 fault。圖中綠線代表 fault 之間的 equivalence 關係，橘色箭頭代表 dominance，而塗紅色的點代表實際放進

list 中的 fault。

■ Fault list: A/1，B/1，C/0，D/0，E/1，G/1，G/0，H/0，I/0

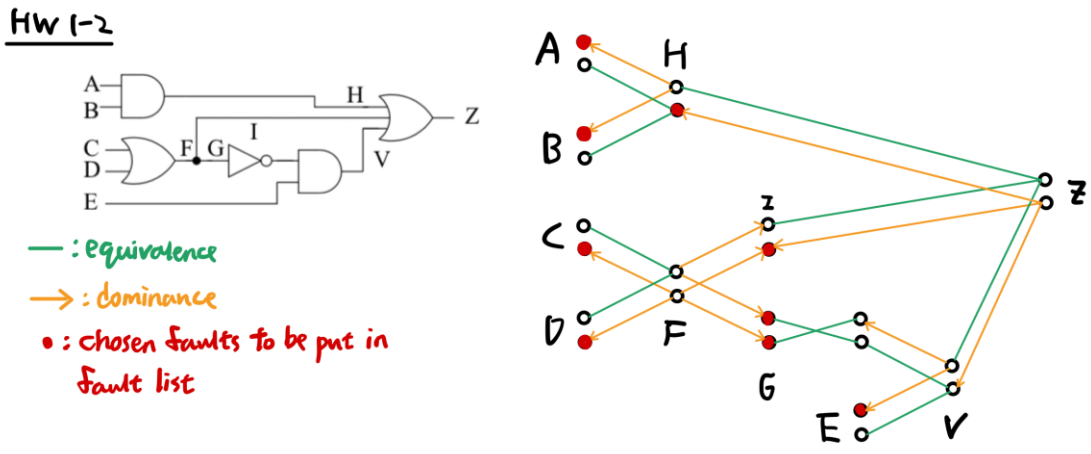


圖 10 Fault diagram

➢ 1-3 Find stuck-at fault

圖 11 為電路之 Verilog code，圖 12 為電路中之節點與 Verilog wire 宣告之對應，而圖 13 則為模擬之波形圖，圖 14 為 testbench。從波形圖中可以看到，在沒有 fault 發生的情況下，*w2* 這條線上的值(紅線)在給定的 5 個 pattern 中都為 0，因此可以知道我們無法利用這五組 test pattern 測試出 *w2*/0 fault (這幾組輸入都不會讓 *w2* 為 1，無法達成 fault activation)，而又因為 *w2* dominate *w1* 與 *A*，因此也無法測出 *A*/0 及 *w1*/0 (無法將 fault propagate 到輸出端)。

```
1  `timescale 1ns/1ps
2
3  module ckt(
4          input A, B, C,
5          output out1, out2
6  );
7
8          wire w1, w2, w3;
9
10         xor u0(w1, B, C);
11         xor u1(out1, A, w1);
12         and u2(w2, A, w1);
13         and u3(w3, B, C);
14         or  u4(out2, w2, w3);
15
16 endmodule
```

圖 11 Verilog code

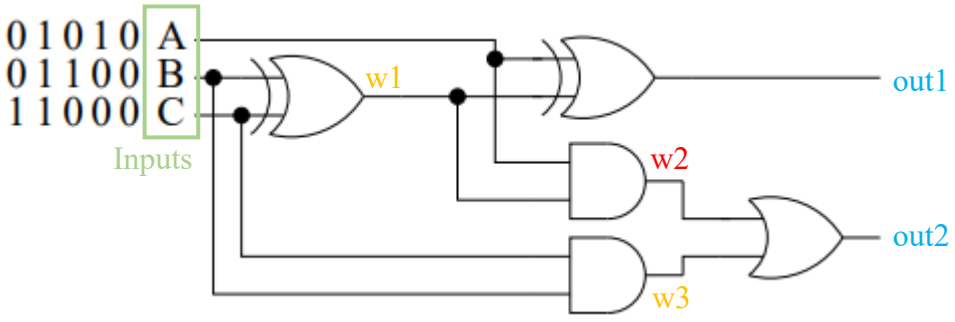


圖 12 電路與 Verilog 之節點對照

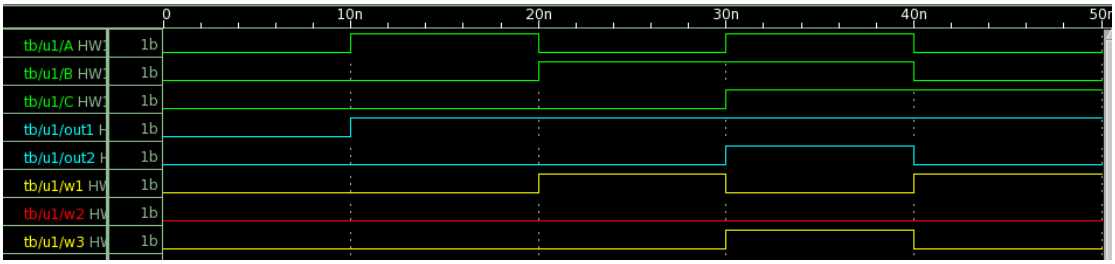


圖 13 電路模擬波形圖

```
1  `timescale 1ns/1ps
2
3  module tb();
4
5          reg A, B, C;
6          wire out1, out2;
7          ckt u1(A, B, C, out1, out2);
8
9          initial begin
10
11         $fsdbDumpfile("HW1_3.fsdb");
12         $fsdbDumpvars(0, tb);
13
14         #0      A = 1'b0;
15                 B = 1'b0;
16                 C = 1'b0;
17
18
19         #10     A = 1'b1;
20                 B = 1'b0;
21                 C = 1'b0;
22
23
24         #10     A = 1'b0;
25                 B = 1'b1;
26                 C = 1'b0;
27
28         #10     A = 1'b1;
29                 B = 1'b1;
30                 C = 1'b1;
31
32         #10     A = 1'b0;
33                 B = 1'b0;
34                 C = 1'b1;
35
36         #10     $finish;
37
38         end
39
40 endmodule
```

圖 14 Testbench